

ICOG LABS TECHNICAL TRAINING

Recommender Systems & Matrix Factorization

Trainer: Naod Ararsa | March 20, 2026

| The Utility Matrix Foundations

The **Utility Matrix** is our starting point. It maps users to items, where each entry is a preference score.

- ✓ **Rows:** Users in the system.
- ✓ **Columns:** Items (Movies, Books, etc).
- ✓ **Sparsity:** The core challenge. We must guess the empty values.
- ✓ **Feedback:** Can be explicit (ratings) or implicit (clicks).



| Explicit vs Implicit Feedback



Explicit

Direct user input. High quality but low volume. (e.g., Star ratings, Thumbs down).



Implicit

Inferred behavior. High volume but noisy. (e.g., Time spent, clicks, purchase history).

Matrix Factorization

THE MODEL-BASED REVOLUTION

The Power of Latent Factors

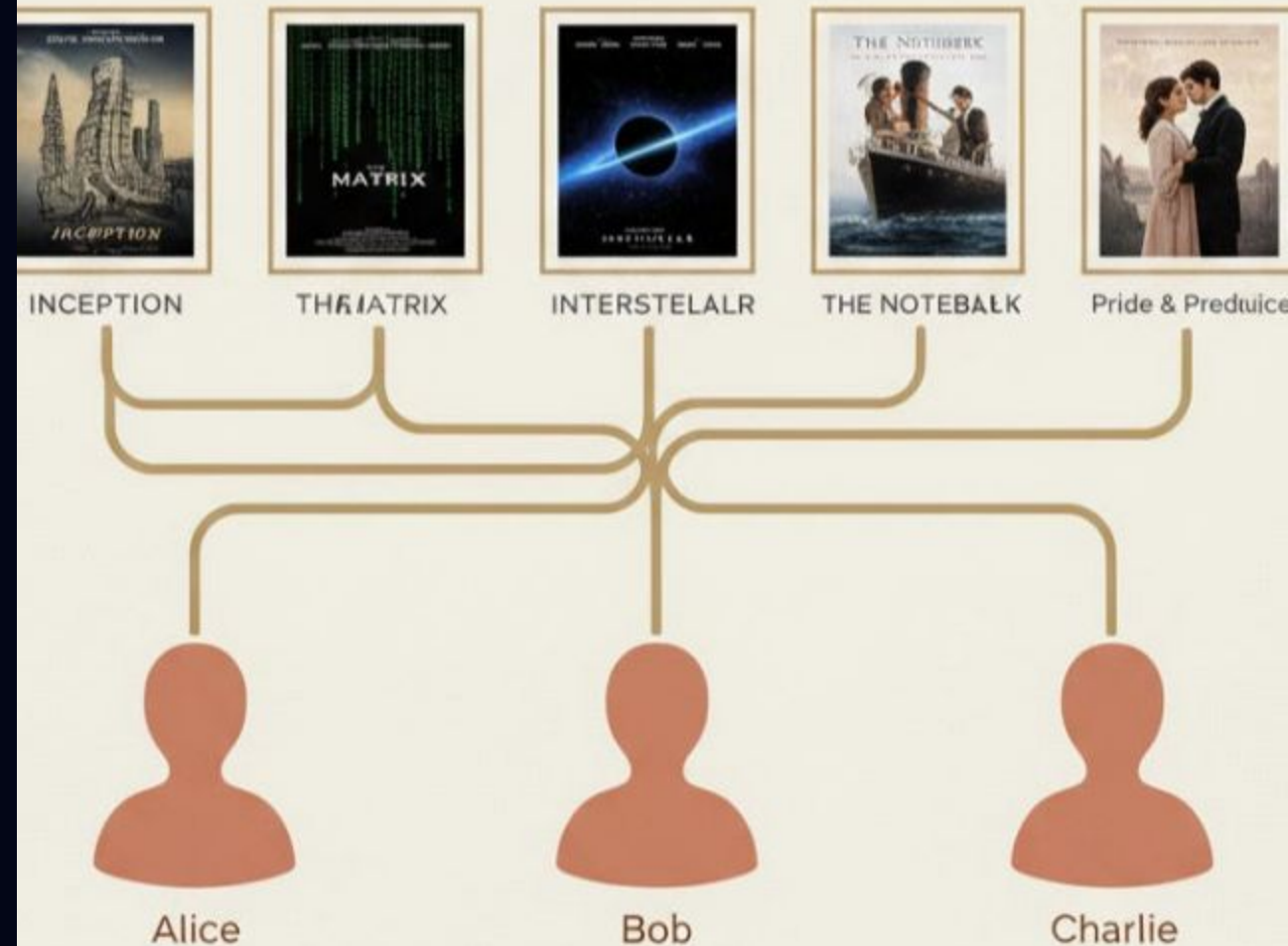
Instead of comparing every user, we decompose the matrix into **Latent Factors**.

These hidden dimensions capture the "essence" of taste, such as a preference for "Surrealism" or "High-speed Action," without explicit labels.

Dimensionality Reduction

We compress thousands of items into a few dozen hidden features.

COLLABORATIVE FILTERING MOVIE RECOMENDATIONS



| Decomposition: U, Sigma, and V

Using Singular Value Decomposition (SVD):

U Matrix

User Profile: Maps Bob to latent factors (e.g., how much Bob likes Action movies).

Σ (Sigma)

Weights: The "Volume Knob."
Importance of each factor in the dataset.

V^T Matrix

Item DNA: Characteristics of the movie (e.g., how much "Action" is in the film).

Putting it Together: Bob's Match

Step-by-Step Prediction

- ✓ **User (Bob):** Affinity for [Action: 0.8, Philo: 0.2]
- ✓ **Weights (Σ):** Dataset importance [Action: 85, Phil: 32]
- ✓ **Item (Matrix):** Features [Action: 0.9, Philo: 0.7]

Result $(0.8 \times 85 \times 0.9) + (0.2 \times 32 \times 0.7) = \mathbf{65.6}$
:

Prediction Logic:

$$R = U_{\text{Bob}} \cdot \Sigma \cdot V_{\text{Item}}^T$$

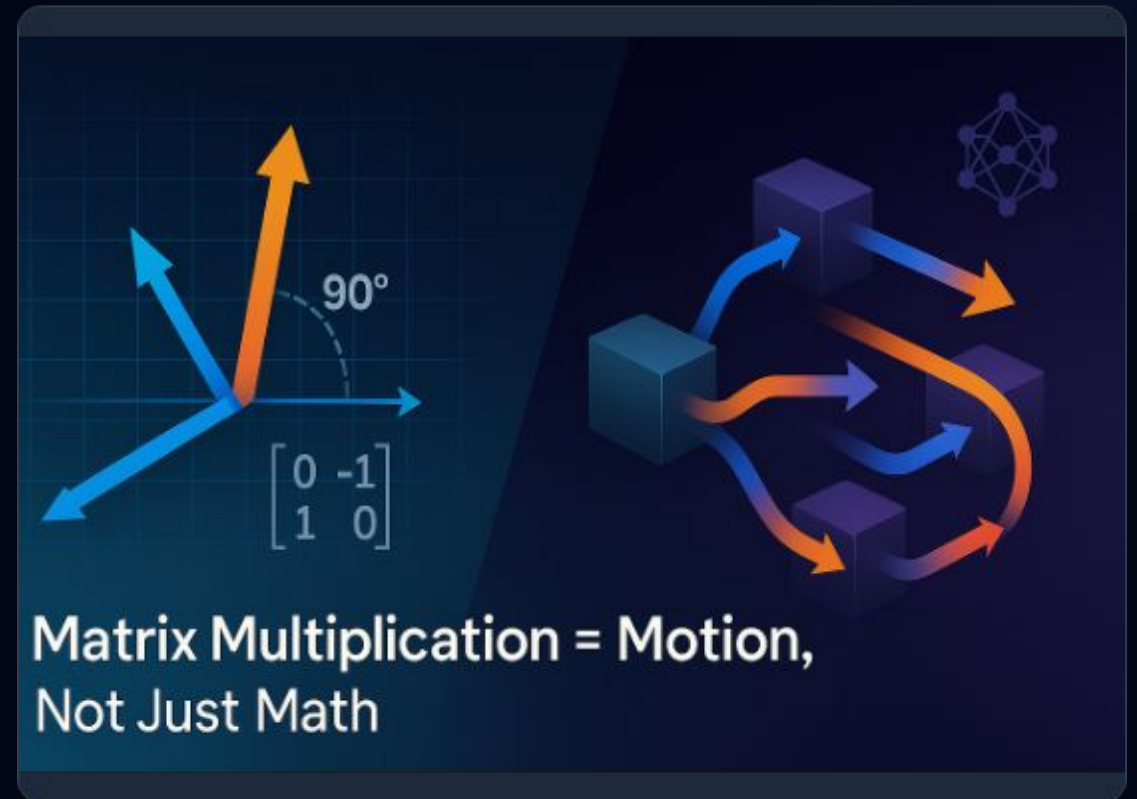
By applying Sigma, we weight the user's preference by the factor's importance.

| How the Computer Learns

Stochastic Gradient Descent

The computer starts with **random guesses** for U and V. It then iteratively reduces the error.

- ✓ Calculate Error: (Actual - Predicted)
- ✓ Nudge U and V values to reduce error.
- ✓ Repeat for thousands of "Epochs".



Evaluation & Challenges

RMSE

Measures how close our predicted "stars" are to reality.
Essential for regression.

Precision@K

Measures the quality of the Top-K list. More relevant for business utility.

Cold Start

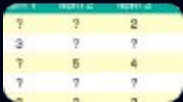
The problem of recommending to new users/items with zero historical interaction.

Questions?

ICOG LABS | AI RESEARCH & DEVELOPMENT

Trainer: Naod Ararsa | naod@icog-labs.com

Image Sources



https://ubc-cs.github.io/cpsc330-2024W1/_images/utility_matrix.png

Source: ubc-cs.github.io



https://miro.medium.com/v2/resize:fit:1400/1*z_hQrDG_QV6tP8YWmQcAsw.jpeg

Source: ai.plainenglish.io



https://miro.medium.com/v2/resize:fit:1200/1*qS6MEF2WxF6cXjZpSUspw.png

Source: medium.com

Practical Project Tasks

Core Requirements

- ✓ **Build System:** Use the **MovieLens** dataset to build a recommendation engine.
- ✓ **Baseline:** Implement a standard Collaborative Filtering model.
- ✓ **Model:** Implement a full **Matrix Factorization** model.
- ✓ **Comparison:** Use **RMSE** to compare model performance.

Bonus Tasks

- ✓ Implement ranking metrics like **Precision@K**.
- ✓ Develop a **Hybrid Recommendation** approach.

| Trainee Evaluation Criteria



Code Quality

Efficiently handling sparse data;
clear, modular structure;
avoidance of inefficient
computations.



Concepts

Explain sparsity, latent features,
implicit vs explicit feedback,
and the cold start problem.



Presentation

Demonstrate results for a
sample user; compare model
performance; discuss design
decisions.